

AI-ORCHESTRATED DEVELOPMENT

Orchestrer plusieurs agents *sans conflits.*

Confier un même projet à plusieurs IA en parallèle est une pratique déjà décrite : un **git worktree** par agent, chacun sa copie du code. Ce qui fait la différence est ailleurs : **le découpage du projet en couches**, et **le cadre qu'on donne aux agents**.



Découper d'abord.

01 · L'ARCHITECTURE

Le projet est d'abord découpé en couches : chacune reçoit un périmètre de code et le type de tâches qu'elle accueille (API, Backoffice, Frontend socle...). Deux tâches envoyées dans deux couches différentes ne travaillent alors jamais sur les mêmes fichiers.



... et autant que le projet en demande : BFF, Frontend espace client, Design system, DevOps...

02 · LA DOCKERISATION

Chaque couche a sa propre stack Docker. Deux couches tournent alors en même temps sans se disputer un port ni écrire dans la base de l'autre, et chacune s'ouvre sur son vhost.



L'architecture donne le cadre
l'agent sait où il a le droit d'écrire

La dockerisation donne le verdict
on ouvre l'URL et on voit le travail tourner

Ce que le découpage permet.

Tout repose sur le découpage de la page précédente. Une couche, c'est une part du code, une branche, un worktree et un vhost. Voilà ce que ça change.

Paralléliser sans conflits

Un worktree par couche. Deux agents n'éditent jamais le même fichier.

Attendre au lieu d'écraser

Un worktree n'accepte qu'un agent à la fois. Le second attend qu'il soit propre.

Dépendre du commit, pas du worktree

On ne rebase pas sur un worktree. Le Backoffice attend donc le commit de l'API.

Vérifier en direct

Une stack Docker et un vhost par couche. On ouvre l'URL, on voit le changement.



Sans cadre, l'agent dérive.

03 · CE QUE L'IA LIT AVANT D'ÉCRIRE

Le cadre n'est pas dans le prompt : il est versionné dans le dépôt, et relu au démarrage de chaque session.

CE QU'ELLE LIT

L'architecture

la carte des couches, le mode de fonctionnement et ce que chacune accueille

CLAUDE.md
docs/
dev-book/
...

CE QU'ELLE LIT

La marche à suivre

un moment du cycle, une commande

/creer-tache
/demarrer-tache
/commit-tache
...

04 · CE QUE L'AGENT NE FAIT JAMAIS

Le cadre ne dit pas seulement où écrire : il dit aussi où s'arrêter. C'est cette moitié-là qui empêche l'agent de partir ailleurs.

JAMAIS

Ouvrir une tâche tout seul

il ne travaille que sur ce qui lui a été demandé, dans cette session

JAMAIS

Reprendre la tâche d'un autre

une tâche non commitée appartient à la session qui l'a écrite

JAMAIS

Annoncer la suite

il ne présente jamais la file d'attente comme la prochaine étape

En échange, il clôt par un inventaire : ce qui est livré, ce qui reste ouvert, ce qui a été croisé au passage.

Six étapes, trois moments, et ça recommence à chaque demande. Ce qui varie d'une demande à l'autre, c'est le nombre de couches traversées — donc le nombre de sous-agents qui travaillent en même temps.

Avant — **Architecturer** · découper le projet en couches isolées

01 La demande instruite par questions, jusqu'à zéro inconnue

02 Le découpage une tâche par couche, jamais un commit partagé

03 La boucle coder, prouver, valider — jusqu'au verdict humain

04 En parallèle couche libre : on avance. Couche occupée : on attend

05 La reconstitution les commits se rassemblent sur une seule branche

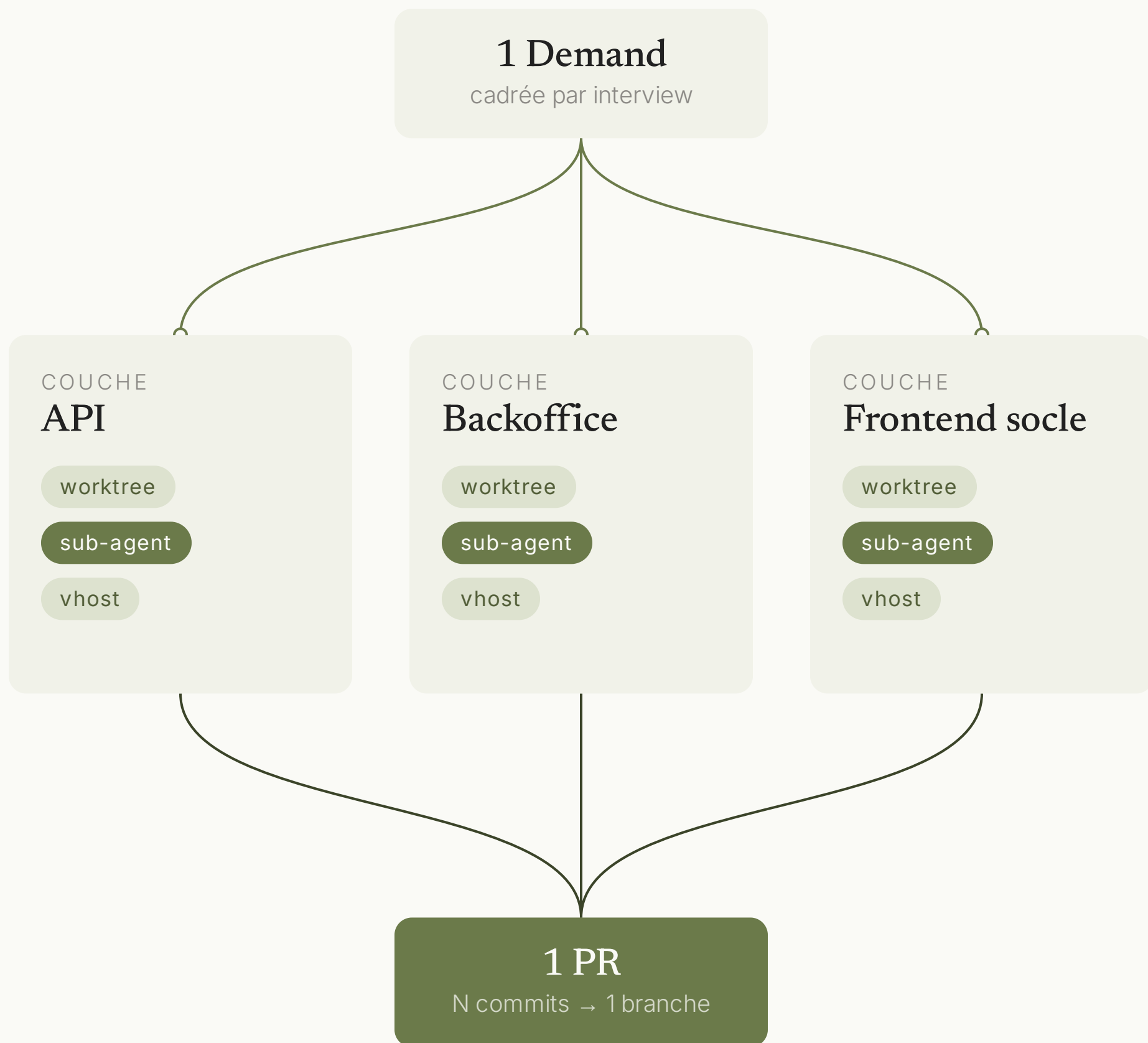
06 La livraison tests, doc, puis PR → CI/CD → merge → deploy

Après — **Consolider** · refactor, patterns, conventions & garde-fous



D'un besoin à une seule livraison.

Une demande, du début à la fin. Elle se répartit entre les couches qu'elle touche ; chacune travaille de son côté, dans son worktree et avec son propre sous-agent ; puis les branches se rejoignent en une seule livraison.



Instruire, puis découper.

01 · LA DEMANDE

Une demande arrive en langage courant. Elle est instruite par questions jusqu'à ce qu'il ne reste plus d'inconnue — aucune décision de cadrage n'est prise à la place de l'humain.

ÉTAPE

L'interview

des questions jusqu'à ce qu'il ne reste plus d'inconnue

/evoquer-un-sujet

ÉTAPE

Le verdict

à faire · à ne pas faire · pas maintenant · pas tranchable

décidé par l'humain

ÉTAPE

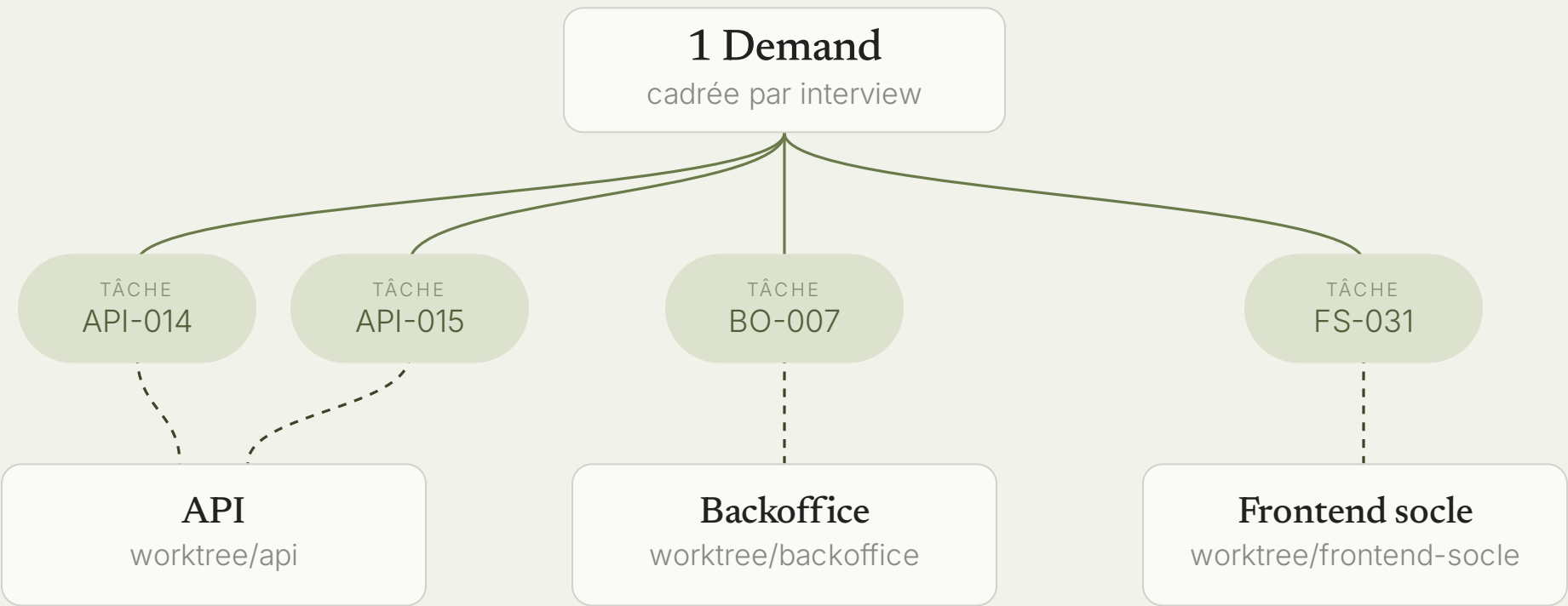
La tâche

ce qu'on fait, ce qu'on ne fait pas, quand c'est fini

dev-book/tasks/

02 · LE DÉCOUPAGE

Une demande donne une tâche, ou plusieurs, sur une couche ou sur plusieurs. Chaque tâche part dans la sienne et n'en sort pas.

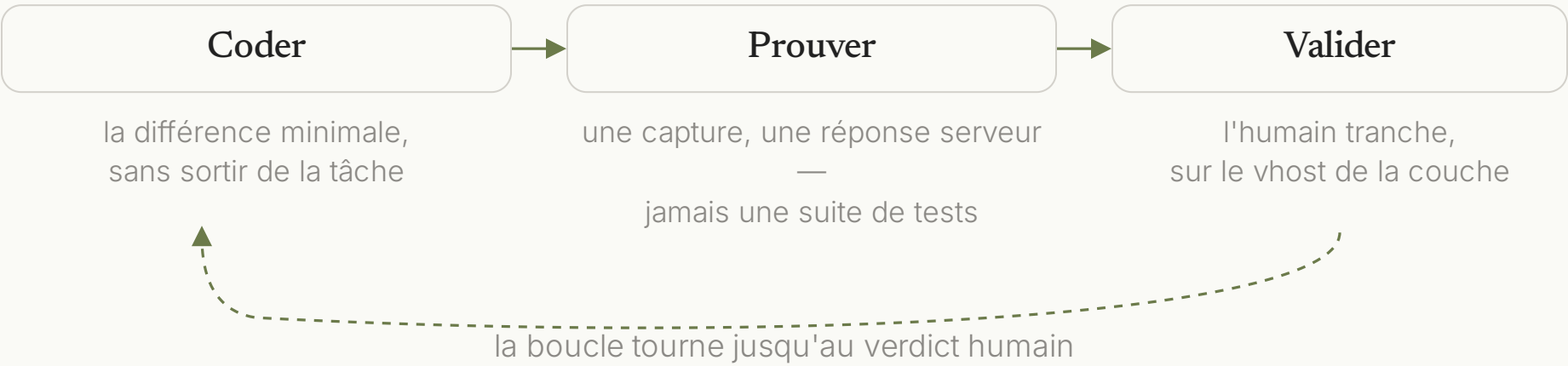


Une tâche, un commit — et deux tâches n'en partagent jamais un.

Coder, puis prouver.

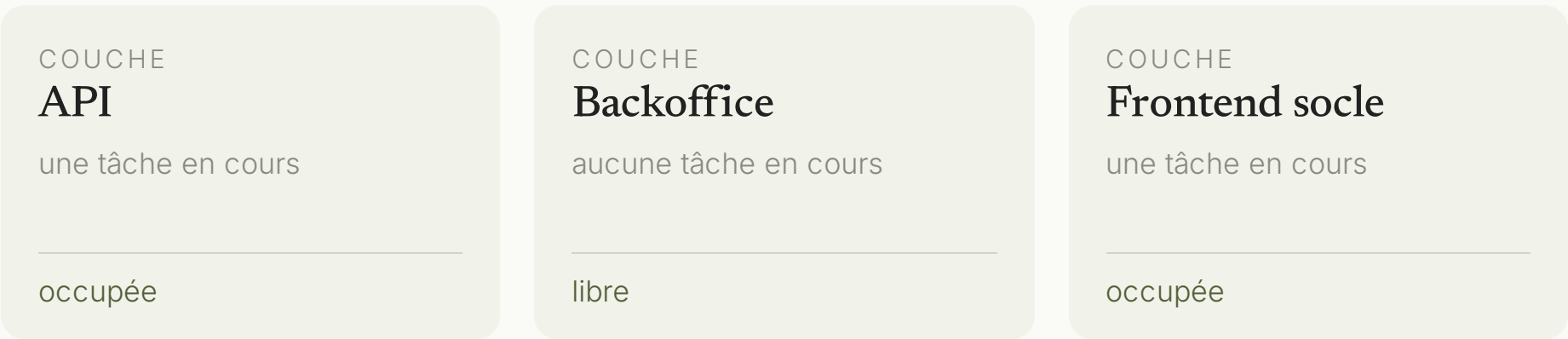
03 · LA BOUCLE

On code ce que la tâche décrit, sans réorganisation opportuniste. Puis on s'arrête sur une preuve fraîche et on attend le verdict humain — la boucle tourne jusque-là.



04 · EN PARALLÈLE

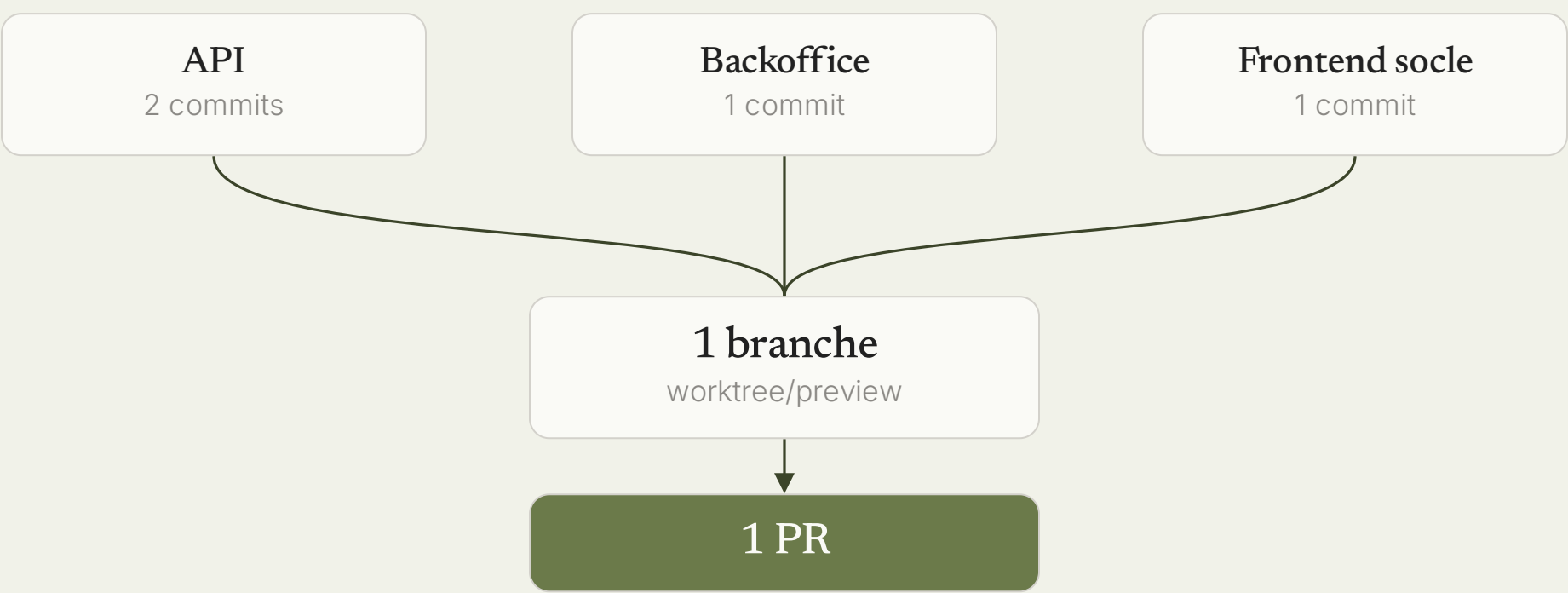
Plusieurs couches avancent en même temps, chacune sur sa stack. Une couche occupée fait attendre le sub-agent suivant plutôt que de se faire écraser.



Rassembler, puis livrer.

05 · LA RECONSTITUTION

Les commits déposés dans les couches se rassemblent par rebase sur une seule branche. Un worktree de preview sert à voir l'ensemble tourner avant d'ouvrir la PR.



06 · LA LIVRAISON

Tests et documentation s'écrivent là, sur l'ensemble. Les contrôles automatiques ne tournent qu'une fois, à l'ouverture de la demande de fusion.



Amélioration continue.

CE QUE LA RELECTURE NE PEUT PLUS ÊTRE

Faire travailler plusieurs agents en parallèle produit plus de code qu'une personne n'en peut relire. À la demande de fusion, on vérifie donc deux choses, et deux seulement : que la tâche fait ce qu'elle annonce, et que le code n'a pas dérivé.

À LA FUSION

Ce qu'on vérifie

la tâche fait ce qu'elle annonce
le code suit toujours la même trajectoire

À LA FUSION

Ce qu'on ne fait plus

lire le code ligne à ligne
rattraper la dette au fil de l'eau

CE QUE LA CONSOLIDATION RATTRAPE

Aucune demande ne déclenche cette phase : c'est une session à part, ouverte quand on décide d'y consacrer du temps. Elle vient après une série de cycles, et on y relit à froid ce qu'ils ont laissé, en cherchant trois choses : ce qui se répète, les erreurs grossières, et les bonnes pratiques qui méritent de devenir la règle.

AMÉLIORATION CONTINUE

Refactoriser

nettoyer ce que la série de
demandes a laissé

clean code

AMÉLIORATION CONTINUE

Relire à froid

patterns, erreurs grossières,
bonnes pratiques

observation

AMÉLIORATION CONTINUE

Poser les garde-fous

conventions, tests, contrôles
CI/CD

le cycle suivant en hérite

Les bonnes pratiques repérées deviennent des conventions techniques.

Les conventions techniques deviennent des garde-fous.

Les garde-fous rendent le cycle suivant *plus sûr*.



Ce qui fait la différence :

l'architecture	→ un périmètre de code et un type de tâches par couche
la dockerisation	→ une stack et un vhost par couche
le cycle IA	→ cadrer, réaliser, livrer — une tâche, un commit
la consolidation	→ les patterns retenus deviennent des conventions techniques

Quatre choses à poser une seule fois. Ensuite, plusieurs agents écrivent en même temps *sans conflits* :

1 Demand = 1 Cycle IA = 1 PR

